

Air Quality Index Forecasting using Machine Learning.

by Warisha Arshad | 10pearls Internship | Data Science Project

Abstract

Air pollution is a serious problem in Karachi, and people often do not know the air quality level in advance. This makes it hard to plan outdoor activities or protect health, especially for sensitive groups. The goal of this project was to build a system that predicts the Air Quality Index (AQI) for Karachi up to three days in advance.

To solve this, a fully serverless machine learning pipeline was built. Live air pollution data (from OpenWeather) and weather data (from Open-Meteo) are collected every hour and stored in a cloud feature store (Hopsworks). Three models were trained and compared: Random Forest, Ridge Regression, and an LSTM neural network (built with TensorFlow). Each model used past AQI values, pollutant levels, and weather patterns to make predictions. The final deployed Ridge Regression model achieved an RMSE of **7.031** on the holdout set. *(Results reported are as of September 6, 2026)*

1 INTRODUCTION

1.1 Problem Statement

Predicting air quality is difficult because pollution levels do not stay constant. They keep changing with weather conditions such as temperature, humidity, wind speed, and pressure, as well as with time-based patterns like the hour of the day, day of the week, and season. Because of this, air quality cannot be predicted using a simple fixed rule — it needs a model that can learn from past patterns. Accurate prediction is important for public health, since people need advance warning to avoid outdoor exposure during hazardous AQI periods, especially in a highly polluted city like Karachi.

1.2 Objective

The objective of this project is to predict the Air Quality Index (AQI) of Karachi for the next 3 days (72 hours) in advance. This is done using a 100% serverless technology stack, so that no dedicated server needs to be managed. The system automatically collects data every hour, retrain its models daily, and serves live predictions through a dashboard and an API.

2 SYSTEM ARCHITECTURE AND DATA PIPELINE

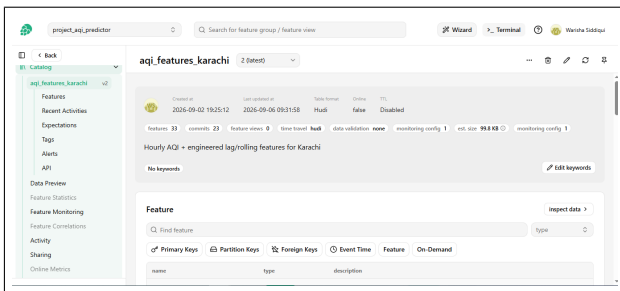


Figure 1: Hopsworks Feature Store Dashboard

2.1 Data Ingestion and Feature Engineering

Raw data is fetched from two external APIs: the **OpenWeather Air Pollution API** for pollutant concentrations

(PM2.5, PM10, NO2, SO2, O3, CO) and the **Open-Meteo API** for weather data (temperature, humidity, wind speed, and pressure) for Karachi. This is implemented in `feature_pipeline.py`. The two sources are merged on a common hourly timestamp, and AQI is calculated using standard breakpoint formulas. We engineered features such as hour-of-day, day-of-week, and month to capture temporal trends, along with lagged pollution values (e.g., AQI and PM2.5 from 1, 3, and 24 hours ago), rate-of-change features, and rolling averages (3-hour, 6-hour, and 24-hour) to help the model capture short-term pollution trends.

2.2 Feature Store Implementation (Hopsworks)

The processed features are stored in the **Hopsworks Feature Store**, in a feature group named `aqi_features_karachi` (version 2), to ensure data consistency and version control. `datetime` is set as the primary key and event time for the feature group. Fig. 1 displays the Hopsworks dashboard showing the successfully created feature groups.

2.3 Historical Data Backfill

A backfill pipeline was executed to populate the feature store with historical data, creating a robust dataset for training. As implemented in `feature_pipeline.py`, if the feature group has fewer than 200 rows, it is automatically treated as “cold” and backfills 30 days of historical pollution and weather data. Once sufficient data is available, the pipeline switches to normal hourly runs, pulling only a small 3-day overlapping window to keep the feature store continuously updated.

3 MODEL DEVELOPMENT

3.1 Feature Selection

Before model training, `select_features()` in `training_pipeline.py` computes the correlation between each candidate feature (pollutant concentrations, weather variables, time-based features, lag features, and rolling averages) and the target variable, `target_aqi_24hr`. Only features with an absolute correlation greater than 0.10 are retained for training; features falling below this threshold are dropped as weak predictors. This step reduces noise and ensures the models are trained

only on features with a measurable relationship to the AQI forecast.

3.2 Model Selection

As implemented in `training-pipeline.py`, three models were trained and compared: **Ridge Regression**, **Random Forest Regressor**, and an **LSTM** (built with TensorFlow). We utilized the Random Forest Regressor for its robustness against overfitting, and TensorFlow (LSTM) to capture sequential time-series dependencies. Ridge Regression was also included as a simpler linear baseline, and it ultimately gave the best result.

3.3 Evaluation Metrics

Performance was measured using **RMSE (Root Mean Square Error)**, **MAE (Mean Absolute Error)**, and **R² Score**. Models were evaluated using time-series cross-validation (`TimeSeriesSplit`) on the training set, followed by a final holdout comparison on unseen data. **Ridge Regression** achieved the best result with an RMSE of **7.031**, and was selected as the final deployed model based on the highest R² score on the holdout set.

Ridge’s regularization strength (α) is automatically tuned via `RidgeCV` over a log-spaced grid (0.01–1000), and Random Forest hyperparameters (`n_estimators`, `max_depth`) are selected via a small grid search — both scored using the same `TimeSeriesSplit(5)` cross-validation used for final evaluation, improving the robustness and generalizability of the reported holdout results.

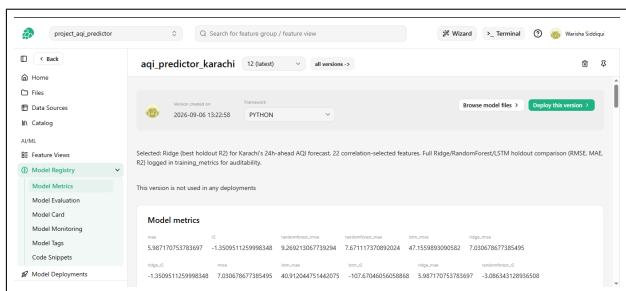


Figure 2: Model Registry

3.4 Model Explainability (SHAP)

SHAP (SHapley Additive exPlanations) values were computed to interpret feature importance and explain individual predictions. This is implemented in `app.py`, which generates SHAP beeswarm and bar plots to visualize how each feature (such as PM2.5, lag values, and rolling averages) impacts the predicted AQI, identifying PM2.5 as a key predictor of air quality.

4 DEPLOYMENT AND AUTOMATION (CI/CD)

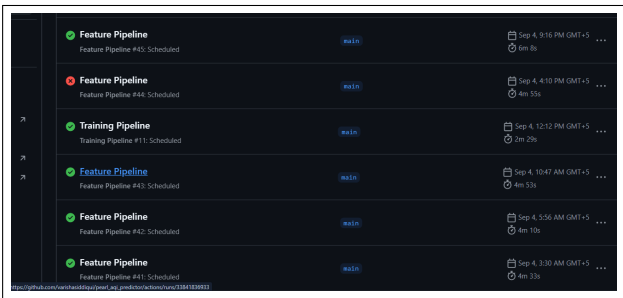


Figure 3: GitHub Actions Workflow Log

4.1 Automated Pipelines

The solution uses **GitHub Actions** for CI/CD, defined in `.github/workflows/`. The feature pipeline runs hourly (`feature-pipeline.yml`), and the training pipeline runs daily at 2:15 AM (`training-pipeline.yml`), both fetching new data and updating the model automatically.

Fig. 3 illustrates the GitHub Actions workflow log, confirming the successful execution of the automated training pipeline.

4.2 Web Dashboard (Streamlit)

A **Streamlit-based dashboard** (`app.py`) was deployed on **Streamlit Cloud** to show the live AQI, the 3-day AQI forecast, and SHAP-based feature importance. The dashboard’s “Historical insights” tab displays the AQI time-series trend, a feature correlation heatmap, and hourly/monthly seasonal patterns — a live, in-app equivalent of the exploratory analysis performed in `eda.py`. Hazardous AQI levels (above 150) also trigger an automated web-hook alert from the feature pipeline, as implemented in `feature_pipeline.py`.

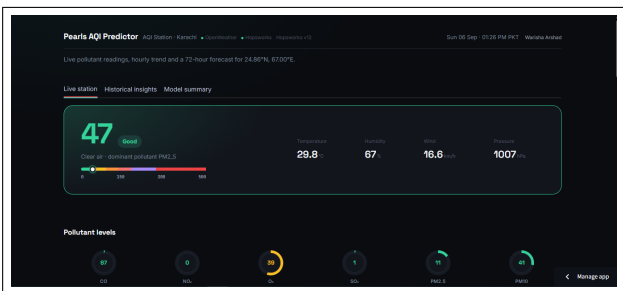


Figure 4: Web Dashboard

4.3 API (FastAPI)

In addition to the dashboard, a standalone **FastAPI** service (`api.py`) was built, running as an independent process from the Streamlit dashboard. It exposes three endpoints: `/health` for a basic health check, `/current` for the live AQI reading, and `/forecast?hours=1-72` for an iterative hourly AQI forecast using the same model and feature logic as the dashboard.

5 CHALLENGES FACED AND SOLUTIONS

5.1 Data Inconsistency and API Limits

Challenge: OpenWeather and Open-Meteo values sometimes returned as whole numbers or decimals, causing data type mismatches with the stored feature schema.

Solution: `feature_pipeline.py` reads the existing feature group's schema at runtime and casts each column to match it before inserting new data.

Challenge: Hopsworks' materialization job sometimes reported a failure even though the data had already been written.

Solution: the pipeline checks Hopsworks' commit history (`commit_details()`) before treating the job as a real failure, avoiding false-alarm pipeline failures.

5.2 Model Drift

Challenge: Karachi's AQI patterns change with weather and season, so a static model can become outdated.

Solution: the training pipeline (`training_pipeline.yml`) is scheduled to retrain the model daily on the latest feature-store data, and the best-performing model (by R^2 score) is automatically re-registered.

6 RESULTS AND ANALYSIS

6.1 Performance Analysis

Three models were compared on a holdout test set — Ridge Regression, Random Forest, and LSTM — using RMSE, MAE, and R^2 Score, as implemented in `training_pipeline.py`. **Ridge Regression** achieved the best result with an **RMSE of 7.031**, and was selected as the final deployed model based on the highest R^2 score on the holdout set.

For context, a naive persistence baseline (predicting the next AQI value as the current AQI) would be expected to yield a considerably higher RMSE, though this baseline was not directly benchmarked in this study.

6.2 Visual Results

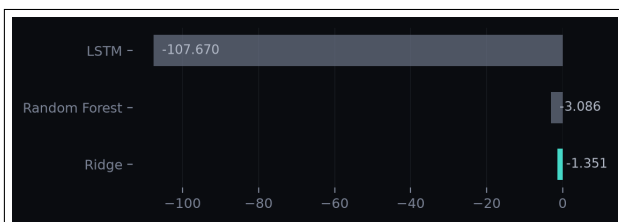


Figure 5: Model Comparison Chart (R^2)

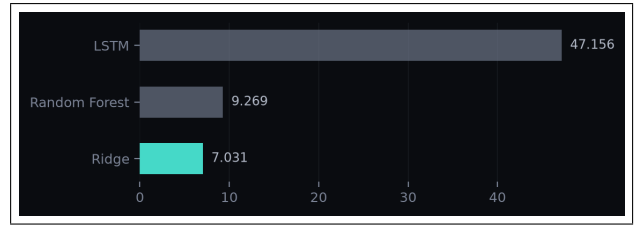


Figure 6: Model Comparison Chart (RMSE)

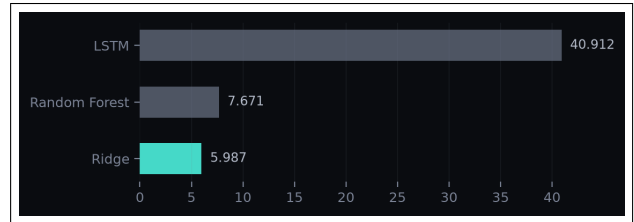


Figure 7: Model Comparison Chart (MAE)

Fig. 5, 6, and 7 show the holdout RMSE, MAE, and R^2 comparison across all three candidate models — Ridge Regression, Random Forest, and LSTM — generated in `app.py`'s "Model summary" tab, with the deployed model's bar highlighted. This visually reflects the holdout comparison described in Section 3.3 (Evaluation Metrics) and shows why Ridge Regression was selected as the final deployed model.

The LSTM's strongly negative R^2 is attributed to insufficient training data for a deep sequential model relative to Ridge and Random Forest, causing it to underfit on the holdout set.

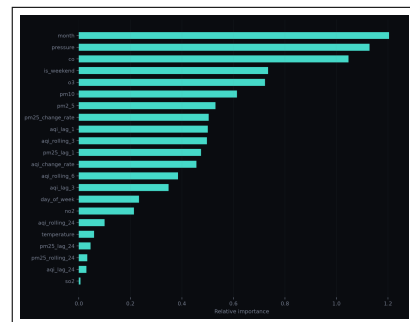


Figure 8: Global Feature Importance Graph

Fig. 8 shows the Global Feature Importance graph, generated in `app.py`, which ranks each feature by its overall contribution to the deployed model's predictions (Ridge coefficient magnitude, or feature importances for Random Forest).

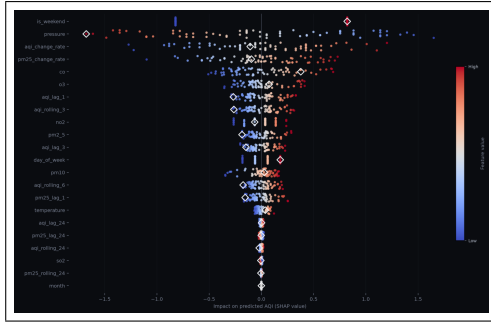


Figure 9: SHAP Summary Plot

Fig. 9 presents the SHAP summary plot for feature importance, generated in `app.py`, which identifies PM2.5 and recent AQI lag/rolling values as the most influential predictors.

7 LIMITATIONS

7.1 Hopsworks Service Dependency

The training pipeline depends on Hopsworks’ Arrow Flight Query Service, with a Hive-based fallback if the primary read fails. In some CI/CD runs, both paths failed due to server-side timeouts (`FlightTimedOutError`, `FlightUnavailableError`), causing that day’s training run to fail.

7.2 No Automatic Recovery for Training Read Failures

Unlike the feature pipeline, which verifies failed jobs against Hopsworks’ commit history before treating them as real failures, the training pipeline has no equivalent recovery step for read failures — a temporary Hopsworks outage directly fails the run until the next scheduled attempt.

7.3 Unhandled Connection Drops During Feature Ingestion

In some runs, the feature pipeline successfully uploaded data to Hopsworks, but the following API call to launch the materialization job failed with a `RemoteDisconnected` connection error, most likely due to a brief server-side connection drop on Hopsworks’ end. Since the pipeline’s recovery logic is designed to catch job-level failures rather than this earlier connection-level error, the run stopped without completing its usual verification step.

7.4 Single-City Scope

The model is trained only on Karachi (fixed latitude/longitude) and does not generalize to other cities or provide hyper-local predictions within different parts of Karachi.

7.5 Linear Feature Selection Bias

Since `select_features()` uses linear correlation ($|r| > 0.10$), it may under-select non-linear predictors relevant to the LSTM model, potentially contributing to its weaker performance.

8 CONCLUSION

The project successfully established a scalable, serverless architecture for AQI forecasting in Karachi, combining automated hourly data ingestion, daily model retraining, and a live dashboard for 3-day forecasts. Future work includes integrating additional spatial data (e.g., multiple monitoring locations) for hyper-local predictions across different areas of the city.